Conception Détaillée
Tétris

Duchesne Lucas

10 mars 2014

Table des matières

1 Introduction **3**

2 Contexte **3**

3 Organisation **3**

4 Description des outils **4**

5 Spécification **4**

6 Conception **4**
6.1 Architecture de conception Modèle - Vue - Contrôleur 4
6.2 Notre implémentation 5

7 La classe main et option **11**

8 La classe Tétris **12**
8.1 La classe Model 13
8.1.1 La classe Tétrominos 14
8.1.2 La classe carre 16
8.2 La classe Controleur 17
8.2.1 La classe souris 17
8.2.2 La classe lecteurSeq 17
8.3 La classe View 18
8.3.1 La classe panel 18
8.3.2 La classe persoPanel 18

9 Conclusion **18**

1 Introduction

Ce manuel décrira comment le code a été pensé pour pouvoir le réutiliser ultérieurement. Ainsi, nous allions étudier en premier lieu la conception générale du tétris grâce à des diagrammes UML afin d' avoir une idée générale du logiciel que nous allons présenter. Par la suite, nous étudierions plus en détail le code pour décrire chaque détail de la programmation du logiciel. Pour cela, nous allons décrire classe par classe le code java et l'expliquer. L'ordre de description des classes sera approximativement celle de l'exécution du programme.

Tetris est un jeu vidéo de puzzle conçu en 1984 par Alekseï Pajitnov. Bâti sur des règles simples et exigeant intelligence et adresse, il est l'un des jeux vidéo les plus populaire au monde.

Des pièces de couleur et de formes différentes descendent du haut de l'écran. Le joueur ne peut pas ralentir ou empêcher cette chute mais peut l'accélérer ou décider à quel angle de rotation (0, 90, 180, 270) et à quel emplacement latéral l'objet peut atterrir. Lorsqu'une ligne horizontale est complétée sans vide, elle disparaît et les blocs supérieurs tombent. Si le joueur ne parvient pas à faire disparaître les lignes assez vite et que l'écran se remplit jusqu'en haut, il est submergé et la partie est finie.

2 Contexte

Tetris est un jeu vidéo de puzzle conçu en 1984 par Alekseï Pajitnov. Bâti sur des règles simples et exigeant intelligence et adresse, il est l'un des jeux vidéo les plus populaires au monde. Ses versions sont innombrables, y compris en 3D, et cette multiplicité se décline sur tous les types d'ordinateurs [Wikipedia] .

Dans un contexte scientifique, l'intérêt de Tetris est qu'il illustre bien les actions dites pragmatiques (e.g. compléter une ligne) tout comme épistémiques (e.g. deviner le type de pièce avant qu'elle ne soit totalement visible via des rotations). Il est de plus utilisé comme tâche cognitive dans plusieurs expérimentations de psychologie et de neurosciences.

3 Organisation

Dans un premier temps, nous avons assimilé les besoins et les enjeux du projet afin de créer un premier modèle UML sur lequel nous avons pu discuté avec notre responsable de projet. Afin de pouvoir avoir un programme fonctionnel en fin de projet, nous avons décider de découper la réalisation du logiciel suivant l'importance des fonctionnalités :

Implémentation du modèle.

Un premier logiciel permettant de déplacer une pièce sur un plateau.

Une première version prototype du logiciel principal permettant de jouer une partie de tétris.

Enregistrer les actions de l'utilisateur (mouvements).

Ajouter le rejeu d'une partie.

4 Description des outils

Le format Comma Separated Values(CSV, valeurs séparées par des virgules en français) est un format représentant un tableau de données de façon visuelle. En effet, chaque ligne de ce type de fichier correspond à une ligne du tableau de données. On repère les différentes colonnes grâce au séparateur (généralement une virgule, ou un point virgule) qui annonce le passage à une autre colonne. Ce format permet donc de représenter simplement des données.

Java et Swing Java est un langage de programmation orienté objet, multiplateforme et possédant une communauté active importante. C'est un langage relativement facile d'utilisation, notamment grâce aux environnements de développ ement intégré notamment NetBean que nous avons utilisé. De plus, il existe beaucoup de bibliothèques internes et externes existantes ce qui permet de gagner du temps lors du développement d'un nouveau projet. Swing est une des bibliothèques internes dont le but est la création d'interface utilisateur graphique (GUI) pour des applications Java. Elle existe depuis de nombreuses années et beaucoup de sites web fournissent de la documentation sur cette bibliothèque. Ces deux technologies ont été imposées par le responsable de projet dès le début. En effet, il fallait garder la possibilité d'étendre le code réalisé et le Java s'y prête bien. Un autre point important était de limiter le nombre de dépendances à des bibliothèques externes afin d'avoir un code le plus standard possible.

5 Spécification

Avant de commencer la description de la conception post programmation du logiciel, nous allons expliciter les spécifications du logiciel voulues dans le cahier des charges.

Nous voulons créer un tétris avec les fonctionnalités normalement établies dans un tétris comme décrit en introduction. Mais nous voulons implémenter une autre version tétris pour des fins d'utilisations pour voir comment les personnes réagissent pendant une partie de tétris. Pour cela, nous voulons pouvoir déplacer les pièces du tétris avec une précision de l'ordre du pixel et une rotation au degré prêt. De plus, nous aimerions pouvoir enregistrer les mouvements d'une partie pour pouvoir la rejouer plus tard, l'analyser, et, pour affecter les décisions des utilisateurs nous voudrions pouvoir tout modifier, tels qu'avoir plus qu'une seule couleur jusqu'à pouvoir commencer le tétris avec des briques déjà en place en bas de l'espace de jeu.

6 Conception

6.1 Architecture de conception Modèle - Vue - Contrôleur

Le MVC est une architecture de conception connue et utilisée dans le monde du dévloppement informatique. Il est l'acronyme de Model-View-Controller (ou

Modèle-Vue-Contrôleur en français). Quand on développe une application graphique, il est important de séparer la partie graphique du reste de l'application afin notamment de pouvoir changer cette partie graphique sans impacter le cour de l'application. Concrètement, le MVC cherche donc à séparer la partie graphique (la vue), la partie données (le modèle) et la partie logique (le contrôleur) de l'application, d'où son nom. On notera que le MVC n'est pas une architecture unique et qu'il en existe un nombre important de variantes. . Dans certaines implémentations, il n'y a aucune interaction entre la vue et le modèle, toute interaction avec le modèle devant être gérée par le contrôleur alors que d'autres permettent à la vue de lire le modèle. Nous allons maintenant voir l'idée générale de ce que représente chacun de ces éléments dans notre vision du MVC.

Le modèle correspond aux données de l'application. Il ne peut être modiffié que par un contrôleur mais peut être lu par la vue. Lorsqu'il change d'état, il informe les vues qui lui sont abonnées.

La vue correspond à ce que voit l'utilisateur, elle est le lien entre l'utilisateur et l'application. Elle présente à l'utilisateur le modèle et transmet les actions de l'utilisateur au contrôleur afinn que celui-ci les traite.

Le contrôleur définit le comportement de l'application. Quand l'utilisateur effectue une action, elle est transmise par la vue au contrôleur qui analyse cette action et va modifier le modèle en conséquence ou changer la vue présentée à l'utilisateur.

6.2 Notre implémentation

Cette conception sera implémentée en trois classe abstract, la classe model sera compensée des fonctions pour gérer la gestion du jeu, la classe view pour afficher les données du jeu, la classe controleur permettant de gérer les événements de l'utilisateur pour le jeu. De plus, nous avons pensé à une classe en plus pour gérer les trois autres tels le montre le graphique suivant :

Le jeu tétris est basé sur la gestion de carré sur un tableau avec des carrés qui forment le décor et d'autres qui forment les pièces du tétris qui tombent. Nous avons donc pensé à gérer ces carrés à l'aide d'une classe carrée permettant de définir les détails de chaque carré de l'écran tels que leurs positions et leurs couleurs. Ainsi pour gérer la pièce descendante, nous avons pensé à une classe tétrominos permettant de gérer cette dernière avec les fonctions s'effectuent dessus et un tableau de carre pour définir la pièce. Et pour définir le tableau de jeu, nous avons défini un tableau de carre dans la classe model.Le jeu a besoin de créer les tétrominos de différents moyens pour cela nous avons pensé à une classe, *terominosFactory* implémentent différentes méthodes, soit pour la création aléatoire des tétrominos soit la création suivant une liste donné en paramètre de notre logiciel.

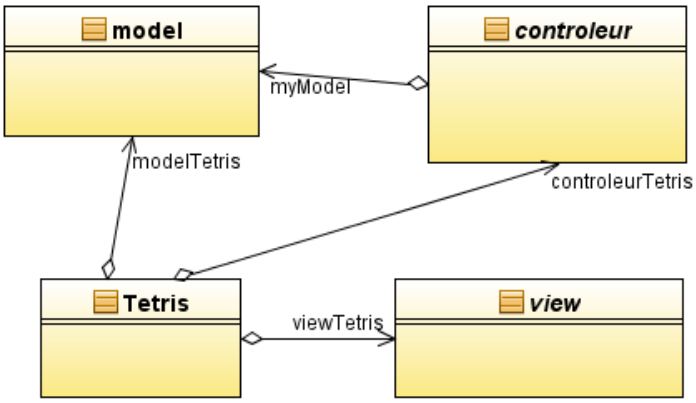

FIGURE 1 – Les classes de base de notre tétris

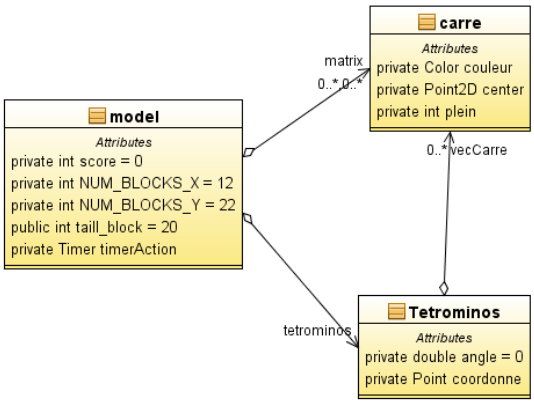

FIGURE 2 – Les classes tétrominos, carre et model

Nous allons donc conçue que le modèle gère un tableau de 12x22, comme le montre la figure suivante, et un tétrominos de 4 carres nous nous fixons dans une échelle ou chaque bloc ferons 20 unités et ainsi nous ne nous occupons pas de l'affichage qui sera remis à la taille voulue lors de l'afficher.

Nous avons conçu le tétrominos pour stocker les données de trois carrés et deux points, l'un pour la rotation et l'autre pour la position du tétrominos sur le tableau de jeu à l'unité prés

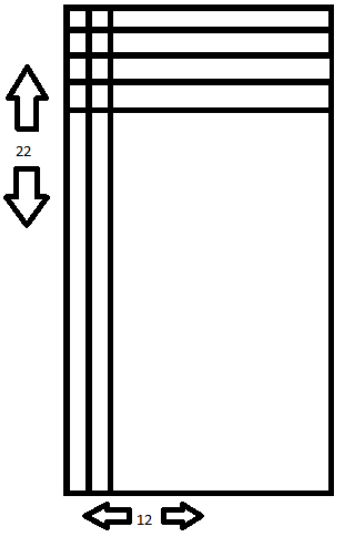

FIGURE 3 – Le tableau tétrominos

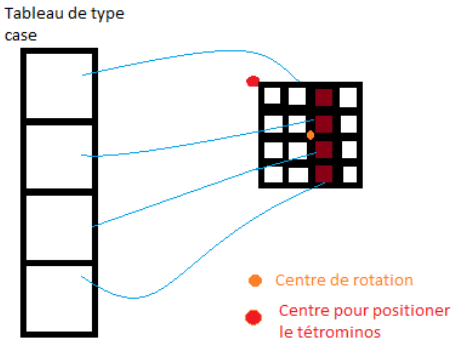

FIGURE 4 – La vecteur carre et la représentation sur le tableau

Pour représenter le déplacement d'un pièce nous suivrons la séquence suivante présentant les étapes lors d'un déplacement selon les X (droite et gauche).

Pour le déplacement du tétrominos selon les Y (vers le bas) sera un appel d'une fonction tous les 100 ms par le timer principal du jeu et lors de l'appel de cette fonction elle suivra ces étapes-là :

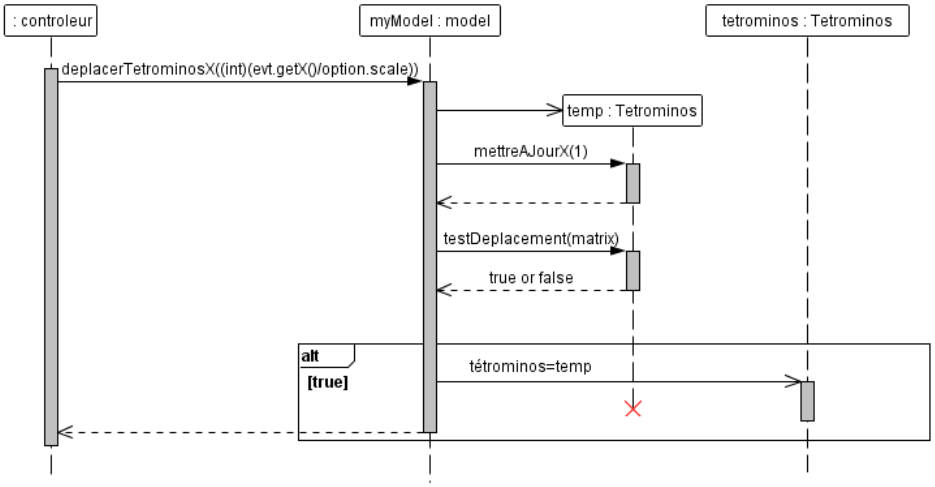

FIGURE 5 – Séquence pour déplacer un tétrominos selon X

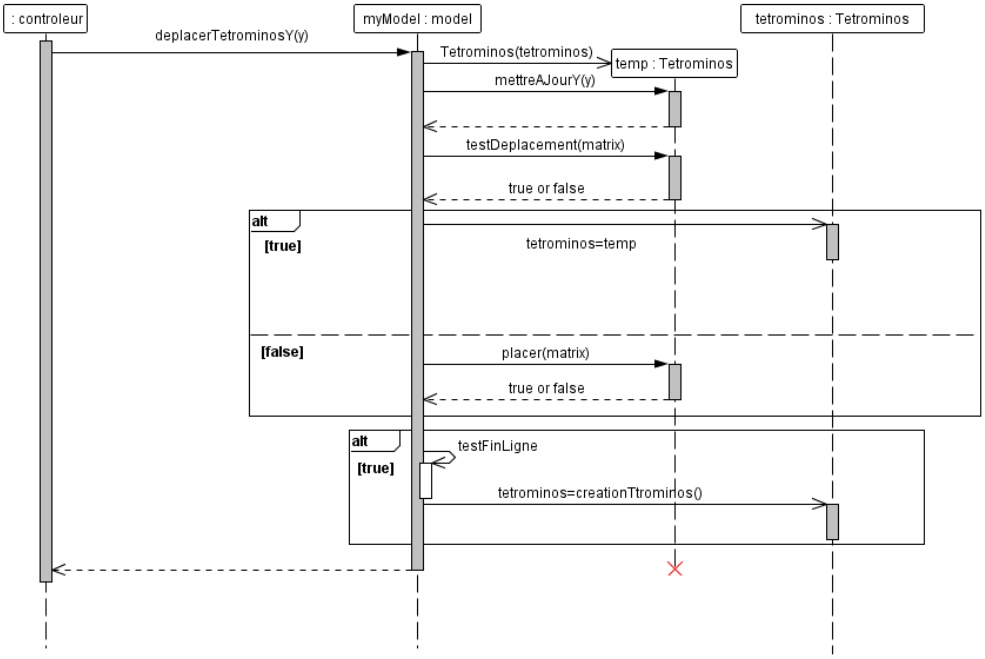

FIGURE 6 – Séquence pour déplacer un tétrominos selon Y

L'affichage choisit pour ce logiciel et l'utilisation de la bibliothèque *Swing* de java pour cela nous avons besoin de réutiliser les classe jFrame et jPanel permettant l'affichage de notre tétris grâce à des fenêtres de base. De plus, ces classes en java permettant de capter les actions de l'utilisateur sur notre logiciel que ce soit mouvement de la souris, touche du clavier...Il faudra donc que notre contrôleur, pour la souris, permette de récupérer ces valeurs et pour la classe view permette d'utilisé le *jPanel*. Nous avons donc décidé de faire la classe principale *tétris* héritant de la classe *jframe* permettant ainsi d'être notre classe principale et de relier le contrôleur, modèle et vue. Les actions, ainsi captées, feront appel aux fonctions abstract de la classe *controleur*.Pour le problème de l'affichage, nous devons réimplémenter la classe *view* et a classe *jPanel*, ici en java, nous avons choisi de faire une classe *panel* héritant de la classe *view* et contenant en variable une classe *persoPanel* héritant de la classe *jPanel*.

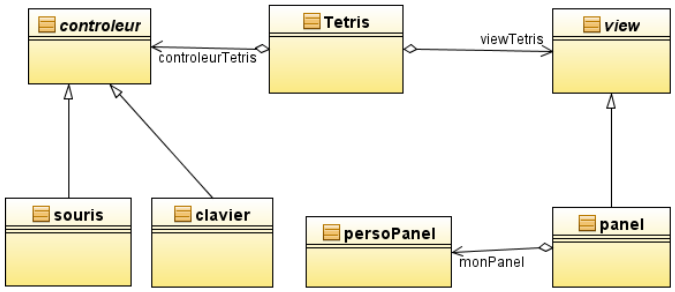

FIGURE 7 – Les classes view et controleur en Swing

Pour notre logiciel, nous avons besoin de sauvegarder les données de jeu pour pouvoir rejouer la partie dans le besoin, pour cela, nous avons besoin d'une classe qui nous permettrons de sauvegarder les actions *savFile* et une classe permettant de les lires *lecterSeq*. Cette dernière sera une classe déduite de la classe contrôleur car nous la considérons comme un contrôleur gérant le jeu comme la souris ou le clavier.

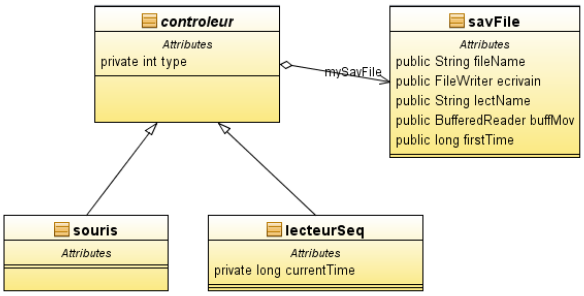

FIGURE 8 – Les classes savFile et lecterSeq

7 La classe main et option

Nous allons ici présenter la classe main, la première classe au début du programme, elle permet de gérer la fenêtre *option* et de lancer une partie de tétris.

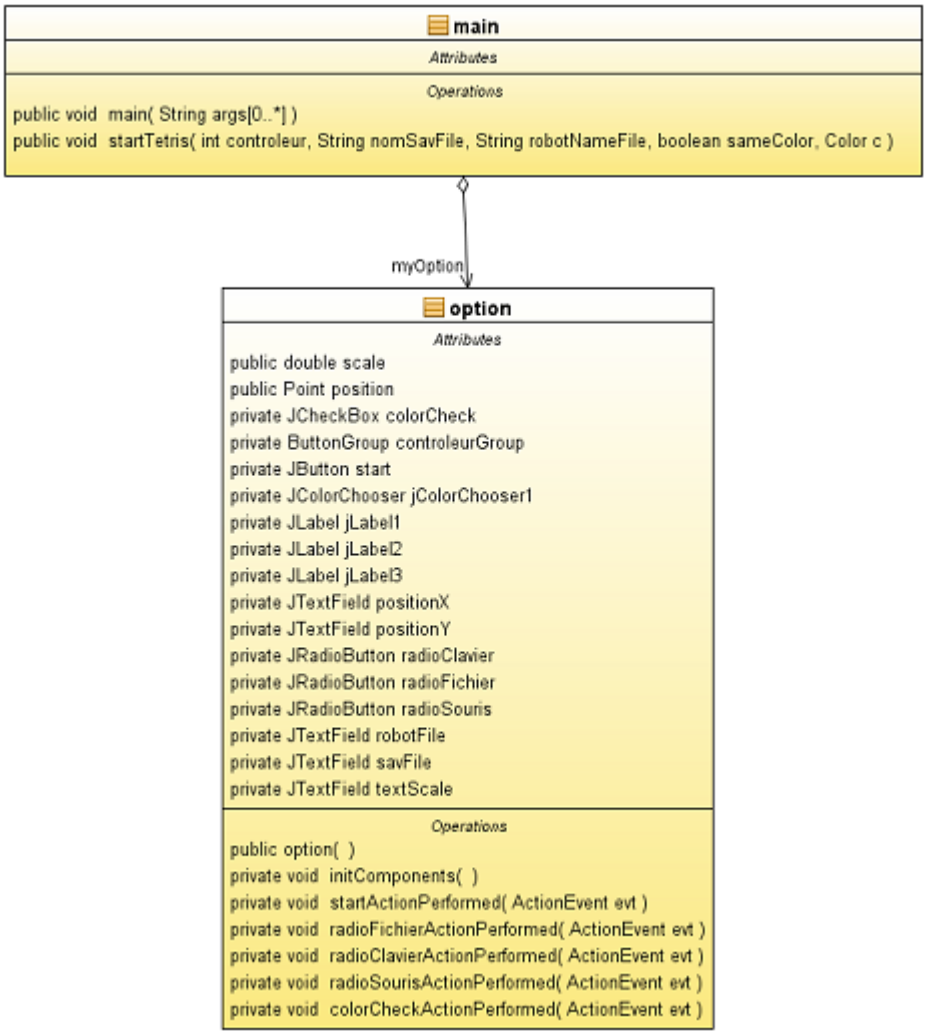

FIGURE 9 – Les classes main et option détaillé

La fonction principale de la classe main et *startTatris* permettant d'intensifier la classe Tétris pour la partie avec les options donnée en paramètre. Elle lance le timer *timerAction* principales de la partie, gérant la chute de la pièce ou la lecture du fichier de sauvegarde.

Nous avons créer la classe *option* héritant de la classe *Jframe* pour pouvoir avoir une interface. Dans cette dernière, nous avons rajouté différents boutons, ratio bouton, des zones de texte (pour la sauvegarde et la lecture de fichier) et le choix de couleur. La fonction principale s'active lors du clic sur le bouton *start* de la fenêtre *option*. Elle appelle la fonction *startTetris* de la classe *main*.

8 La classe Tétris

Nous avons vu à la conception que cette classe permet de faire la liaison entre les différentes classes du modèle MVC en étant en java nous devons créer une classe *jFrame* pour avoir accès à l'affichage et à la capture des événements. Nous avons donc implémenté cette classe telle que suit :

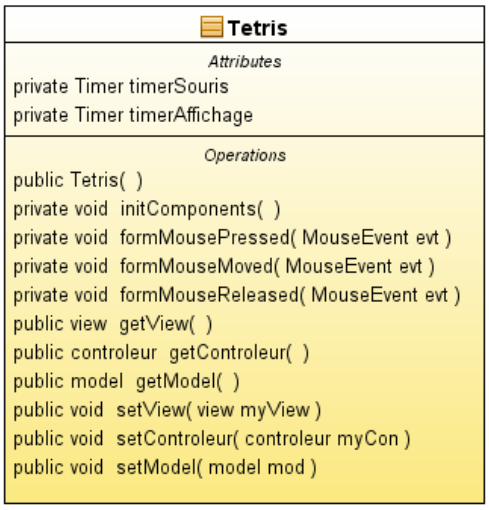

FIGURE 10 – La classe Tétris

De ce fait, l'héritage de la classe *Jframe* pour permettre d'avoir un affichage en swing que l'on utilisera dans la classe view. Elle a comme attribut un modèle *modelTetris*, une view *viewTetris*, un model *controleurTetris* et ainsi que le timer d'affichage *timerAffichage* et le timer de la souris *timerSouris* qui sert à effectuer une action pendant le temps ou l'utilisateur presse un bouton de la souris.

Le *timerAffichage* va nous servir de timer pour rafraîchir notre écran d'affichage pour cela elle va appeler la fonction *afficher* de la classe *view* tous les 50 ms pour afficher le jeu à l'écran.

8.1 La classe Model

La classe *model* permet de gérer le jeu par rapport aux éléments reçu par la classe *controleur* et de stocker les données du jeu tétris. Nous allons parlez d'abord des attributs de cette classe et ainsi de comment sont stockées nos données de jeux. Pour cela nous avons stocké le tableau de jeux dans un tableau de classe *carre* à deux dimensions de taille fixe permettant de décrire le plateau de jeu appelé *matrix*. On stock une instance de la classe *Tétrominos* pour savoir quelle pièce on utilise et où la positionner.Le timer stocké *timerAction* permet de gérer la descente de la pièce lors du choix du jeu à la souris ou autre contrôleur ou d'effectuer la prochaine action lors de la lecture du fichier enregistré.

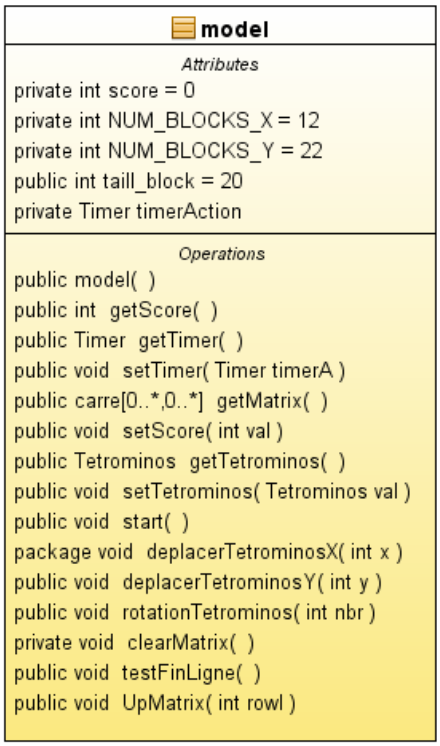

FIGURE 11 – La classe model

La classe *model* a les principales fonctions du jeu :

DeplacerTetrominosX : permet de déplacer le *tétrominos* courant, ou plutôt la coordonnée haut gauche du *tétrominos* en considérant que le *tétrominos* est dans un carré de 4x4 carrés. Pour cela nous appelons la fonction *mettreAJourX* de la classe *tétrominos*.

DeplacerTetrominosY : permet de déplacer le *tétrominos* courant selon les coordonnées X et tests si on peut encore déplacer la pièce dans le cas contraire appel la fonction *placer* de *tétrominos*.

rotationTetrominos : permet de faire la rotation de la pièce courante en testant si la rotation peut se faire dans ce cas-là, on appelle la fonction *rotation* de la classe *tétrominos*.

testFinDeligne : permet de tester s'il existe des lignes complètes et de les supprimer du tableau de jeu. Pour cela, elle va tester toutes les lignes en partant du bas jusqu'à ce qu'elle trouve une ligne complète.

8.1.1 La classe Tétrominos

La classe *Tetrominos* permet de gérer chaque pièce et de stocker leur position sur le plateau. Nous avons donc implémenté cette classe pour pouvoir stocker les données sur le tétrominos en cours de jeu et les actions possible dessus. Nous stockons donc pour cela dans la variable *vecCarre* une liste composée de 4 carrés. En lisant chaque carré de cette liste nous pourrons reconstruire notre tétrominos, mais pour savoir ou le placer sur le plateau nous stockons sa positon par un seul point fictif, *coordonne*, en haut à gauche (cf figure 4) ainsi pour tester pour savoir ou est positionné chaque carrée du tétris.

Nous avons donc $\begin{pmatrix} Co_x + 2 * tb \\ Co_y + 2 * tb \end{pmatrix}$ pour le centre de notre tétrominos avec $Co = \begin{pmatrix} Co_x \\ Co_y \end{pmatrix}$ notre point fictif et tb la taille d'un bloque.

De plus nous stockons l'angle de rotation du tétrominos, *angle*, pour pouvoir l'utiliser lors de l'affichage ou autre.

La classe a comme fonction principale, *testDeplacement* qui permet de tester si le tétrominos peut se déplacer à cette position sur le plateau. Pour cela, elle va tester pour chaque *carre* du plateau si elle touche une *carre* du tétrominos actuelle avec la fonction *intersects*, de la classe *rectangle*, implémenté par la classe *carre*. Cette fonction permet de test l'espace (en pixel) du rectangle appartient à l'autre rectangle. Du fait notre algorithme devient :

Listing 1 – la boucle pour le test du déplacement

```
for (carre[] matrix1 : matrix) {
     for (int col = 0; col<matrix[0].length; col++) {
      //pour chaque elements de notre tableau :
       for (carre c : vecCarre) {
         //pour chaque carre de notre
         // vecteur du tetrominos :
         if (matrix1[col].getBound().intersects(
            c.getBound())&& matrix1[col].getPlein() == 1)
         {  //si les deux rectangle se touche renvoyer :
            // impossible de se deplacer ici
            return false;
         }
       }
   }
}
```

La fonction *placer* est une fonction principale qui permet de placer la pièce sur le plateau de jeu. Pour cela, en premier lieu elle remet la pièce à l'endroit si la pièce n'est pas dans une rotation normale du jeu tétris nous testons ceci grâce une différence et des tests :

Listing 2 – Remise du tétrominos dans une position normal du tétris

```
temp=angle;
//on recupere l'angle modulo 2Pi
double angl= angle%(2*Math.PI);
//on test dans quel partie est notre angle
if(angl<0){
    if(angl<-7*Math.PI/4 || angl >= -Math.PI/4)
  {
      temp=-temp;
  }
  else if(angl<-Math.PI/4 && angl>=- 3*Math.PI/4)
  {
      temp=-temp-Math.PI/2;
 ....
}
//on effectue la rotation nesseraire
//pour replacer notre tetrominos
this.rotation(1,temp);
```

Après rotation de la pièce on vérifie si nous ne pouvons pas déplacer la pièce encore dans ce cas la, nous rendons la main à l'utilisateur. Dans l'autre cas, on met les *carre* de la pièce dans le tableau à l'endroit où sont les *carre* de *vecCarre*.

Les autres fonctions *mettreAJourX*, *mettreAJourY* et *rotation* permet de déplacer et d'effectuer la rotation du tétrominos pour cela nous parcourons

chaque carre du vecteur et nous déplaçons les coordonnées de chaque *carre* ou de faire la rotation avec la fonction *rotationCenter* de *carre*.

La classe tetrominosFactory Cette classe permet de gérer la création des tétrominos de différents moyens possibles. Nous avons créé la fonction de telle façon qu'elle gére la gestion de différents types de création pour savoir quel type de création nous stockons dans la variable *modeCreation*. Les différents types de création ont leur propre fonction, pour une création aléatoire *randTetrominos*, nous devons préalablement spécifier un fichier de sauvegarde de la séquence pour pouvoir rejouer la partie avec la fonction *setNameFileSav* et pour la création par rapport à une séquence enregistré *seqTetrominos*. Pour cette dernière nous devons spécifié le nom du fichier à lire avec la fonction *setNameFileSeq*.

La fonction *setTetrominos* permet de créer un tétrominos du type passé en paramètre et le renvoie.

8.1.2 La classe carre

Cette classe permet de gérer chaque case du jeu. Pour cela, elle hérite de la classe *rectangle* pour avoir facilement la gestion graphique de carré et des déplacements des cubes sur le jeu. Nous stockons en plus comme variable la couleur du cube *couleur*, une variable pour savoir si le carré est plein *plein* et une variable de type *Point2D* pour stocker le centre du carre.

Pour gérer les déplacements, nous avons besoin de savoir quel espace occupe chaque carre sur notre modèle pour pouvoir tester par la suite avec les autres éléments. Nous avons donc crée une fonction *getBound* nous renvoyons l'espace occupé, moins un pixel de chaque coté pour facilité la détection des collisions, par notre carré suivant cette fonction pour crée le carré :

De centre $\begin{pmatrix} 1 + C_y - tb/2 \\ 1 + C_x - tb/2 \end{pmatrix}$ et de taille $tb - 2$ avec $C = \begin{pmatrix} C_x \\ C_y \end{pmatrix}$ centre du carrée et $tb = 20$ unités la taille d'un bloque.

Les fonctions principales de cette classe sont :

rotationCenter : permet de faire la rotation du *carre* grâce à la fonction *AffineTransform* qui va effectue la transformation de la variable *center*.

AffineTransform utilise les rotations de matrice avec un angle θ pour un point donnée (*center*) $Cen = \begin{pmatrix} Cen_x \\ Cen_y \end{pmatrix}$ avec au préalablement fait une translation à l'origine $\begin{pmatrix} 0 \\ 0 \end{pmatrix}$:

$$Cen_{new} = \begin{pmatrix} cos(\theta) & -sin(\theta) \\ sin(\theta) & cos(\theta) \end{pmatrix} \begin{pmatrix} Cen_x \\ Cen_y \end{pmatrix}$$

changerCoordonne : permet de faire la translation du *carre* grâce à la fonction *translate* de la classe *rectangle*. Nous déplaçons chaque carré du tétrominos grâce à cette fonction en envoyant en paramètres les d_x et d_y.

changeCarre : permet de déplacer un *carre* lorsque on veut descendre une ligne.

8.2 La classe Controleur

La classe *controleur* permet d'implémenter les différents contrôleurs du tétris plus facilement. Nous avons développé cette classe de façon d'avoir en variable la classe *savFile*, contenant le nom du fichier de sauvegarde des commandes et permettant d'effectué des actions dessus. Elle garde aussi en variable le type de contrôleur utilisé lors de la section dans la variable *type*.

La principale fonction est *gestionTouche* qui est abstract et qui permet de l'utiliser lors du développement des différents types de contrôleur.

8.2.1 La classe souris

Cette classe implémente la classe *controleur*, elle permet de gérer les déplacements de la souris. Comme nous avons développé le programme sous *swing* pour capter les mouvements de la souris nous avons mis dans la classe *tétris*, étant notre *Jframe* principale, les fonctions de swing pour capté ces événements. Ces fonctions sont les suivantes dans la classe *tétris* : *formMousePressed*, *formMouseMoved* et *formMouseReleased*.

La fonction *formMouseMoved* permet de récupérer les coordonnées de la souris lors du déplacement et de l'envoyer à la fonction *gestionTouche* de la classe *souris*. Cette dernière va appeler la fonction *deplacerTetrominosX* du *model* avec en paramètres selon la position X de la souris.

Les fonctions *formMousePressed* et *formMouseReleased* permettent de gérer le clique de la souris. La première s'activera lorsque que nous cliquerons sur un des boutons (bouton droit permet de tourner à droite et inversement pour le gauche), et déclenchera un timer effectuant dans chaque cas l'appel de la fonction *rotation* du *model* et sauvegardera le mouvement dans le fichier de la classe *savFile* tous les $100ms$, nous pourrons changer cette valeur pour avoir une plus grande vitesse de rotation pour l'instant la vitesse est de $0.1deg/ms$.

8.2.2 La classe lecteurSeq

Cette classe implémente la classe *controleur*, elle permet de gérer la lecture d'un fichier de jeu et d' effectuer la partie enregistrer. Pour cela, elle va charger le fichier voulu dans la classe *savFile* lors de la création de l'instance de la classe.

J'ai implémenté la fonction *gestionTouche* de la classe *controleur* pour quel gère la lecture du fichier, pour cela elle appelle la fonction *getNextMove* de la classe *savFile* qui permet de récupérer la prochain mouvement écrit dans le fichier sauvegarde et d'appeler la fonction correspondante dans la classe *model* pour quel rejoue la partie sauvegardée.

Cette fonction implémentée, sera appelée dans le *timerAction* tous les 10 ms, comme nous avons sauvegardé le temps pour chaque action et nous pourrons modifier cet appel pour que cela puisse effectuer le jeu sauvegardé en temps normal en effectuant toutes les actions passées entre chaque 10 ms.

8.3 La classe View

La classe *view* est une classe permettant d'implémenter les différentes vues de notre *tétris* dans le cas où nous voudrons avoir différents affichages. Cette classe a comme fonction *afficher* permet d'afficher le tétris. Dans notre cas, nous l'avons implémenté pour qu'elle puisse utiliser la bibliothèque *Swing* du java.

8.3.1 La classe panel

Pour implémenter l'affichage en *Swing* nous avons besoin d'implémenter la classe *JPanel* et aussi *view* pour notre programme. Pour cela, cette fonction fera la liaison entre l'implémentation des deux. Elle a comme variable *monPanel* de type *persoPanel*, notre implémentation du *JPanel*.

afficher est implémenté dans cette classe pour appeler le *repaint* de la classe *persoPanel*

8.3.2 La classe persoPanel

Cette classe permet d'implémenter la classe *jPanel* pour avoir un affichage en *swing*. Cette classe a comme variable *matrix* et *tetrominos* pour savoir ce qui faut afficher et la variable *sameColor* qui permet de savoir si l'affichage doit se faire avec une seule couleur.

9 Conclusion

Au cours de ce projet, nous avons pu élaborer un tétris continue. Le logiciel devra permettre à terme de faire des testes sur des utilisateurs pour analysé leur réaction par rapport au jeu pour en déduire des réactions des personnes. Le développement modulaire et l'utilisation de technologies standards permettront au logiciel d'être évolutif et de s'adapter aux contraintes réelles qui n'ont pas pu à ce jour être vérifiées. Ce projet nous a tout d'abord permis d'avoir une expérience de développement Java. Il nous a aussi permis de nous intéresser de façon approfondie au gestion de déplacement continue d'une figure. Si le logiciel est aujourd'hui dans une version avancée, il reste néanmoins certaines fonctionnalités qui n'ont pas pu atteindre leur but.

En conclusion, le programme implémente les fonctions essentielles du cahier des charges :

1. Reproduction du comportement du jeu standard
   (a) Mêmes pièce, les 7 pièces différentes, **Oui**.
   (b) Mêmes actions = tourner/déplacer/tomber, **Oui**.
   (c) Possibilité de mettre en pause, **Non**.
2. Déplacement avec une précision arbitraire des pièces, **Oui**.
3. Rotation avec précision arbitraire, **Oui**.
4. Contrôle du jeu à la souris, **Oui**, la classe *souris*.

5. Contrôle du jeu au clavier, **Non**, classe existante mais pas implémentée.
6. Gestion des conflits entre commandes de l'utilisateur et contraintes du jeu, **Oui**, la fonction *tetrominos.testDeplacement()*.
7. Recaler les Tetrominoes sur la grille des blocs une fois qu'ils sont stabilisés/tombés, **Oui**, la fonction *tetrominos.placer()*.
8. Détermination de la séquence des Tetrominoes, **Oui**, la classe *tetrominosFactory*.
9. Sauvegarde du comportement de l'utilisateur, **Oui**, la classe *savFile*.
10. Contrôle du jeu directement depuis un fichier de données, **Oui**, la classe *lecteurSeq*.
11. Limitation possible de la vitesse maximale de déplacement des Tetrominos, **Oui**.

Le code correspond aux exigences techniques voulues :

1. Implémentation en Java, **Oui**.
2. Interface graphique Swing, **Oui**.
3. Finalisation sous forme de package exécutable avec code source intégré, **Oui**.
4. Indépendance à des bibliothèques et les packages sont standards , **Oui**.
5. Modèle de conception permettant de produire plusieurs vues/contrôles, **Oui**.
6. Rendre le code le plus modulaire et extensible possible, **une grande partie**.